

SBM-L Control Chart

1 SBM-L Control Chart

1.1 Stage3A_SBM_L Function

```
library(readxl)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(tidyr)
library(ggplot2)
library(purrr)
```

Warning: package 'purrr' was built under R version 4.4.3

```
set.seed(123)
setwd("C:/Users/xyang1/CV Research Paper/stability paper")
file="C:/Users/xyang1/CV Research Paper/stability paper/Equipment_A.xlsx"
```

```

stage3A_SBM_L <- function(file,
                           target_ARL0 = 740,
                           B = 10000,
                           c_smooth = 0.5,
                           seed = 123,
                           event_col = "Event",
                           location_col = "Location",
                           result_col = "Result",
                           lod_col = "LOD",
                           loq_col = "LOQ",
                           usl_col = "USL",
                           plot_chart = TRUE) {

  required_pkgs <- c("readxl", "dplyr", "tidyr", "ggplot2", "purrr")
  missing_pkgs <- required_pkgs[!(required_pkgs %in% installed.packages()[, "Package"])]

  if (length(missing_pkgs) > 0) {
    stop(
      "Please install the following packages first: ",
      paste(missing_pkgs, collapse = ", ")
    )
  }

  library(readxl)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
  library(purrr)

  set.seed(seed)

  # ARL0-based calibration

  if (target_ARL0 <= 1) {
    stop("target_ARL0 must be greater than 1.")
  }

  alpha <- 1 / target_ARL0

  # Step 0: Identify available sheets

```

```

all_sheets <- readxl::excel_sheets(file)
residue_types <- c("DAR", "CAR", "Mic")
available_types <- intersect(residue_types, all_sheets)

if (length(available_types) == 0) {
  stop("No valid sheets found. Sheet names must include at least one of: DAR, CAR, Mic")
}

if (length(available_types) == 1) {
  stop(
    "Only one residue-type sheet was found: ",
    available_types,
    ". Please use the univariate control chart method instead of the multivariate SBM-
  )
}

message("Available residue types used in SBM-L chart: ",
  paste(available_types, collapse = ", "))

message("Target ARL0 = ", target_ARL0,
  "; alpha = ", signif(alpha, 5))

# Helper function: detect ND symbol

is_nd_result <- function(x) {
  x_chr <- trimws(toupper(as.character(x)))
  x_chr %in% c("ND", "NON-DETECT", "NONDETECT")
}

# Step 1 and Step 2:
# Read each sheet, replace ND by LOD or LOQ, normalize by USL

read_and_normalize <- function(type) {

  dat <- readxl::read_excel(file, sheet = type)

  # Choose actual result column.
  # Usually CAR and Mic use "Result".
  # DAR may use "DAR" as the result column.
  result_col_actual <- result_col

```

```

if (!(result_col_actual %in% names(dat)) && type %in% names(dat)) {
  result_col_actual <- type
}

required_cols <- c(
  event_col, location_col, result_col_actual,
  lod_col, loq_col, usl_col
)

missing_cols <- setdiff(required_cols, names(dat))

if (length(missing_cols) > 0) {
  stop(
    "Sheet ", type, " is missing required columns: ",
    paste(missing_cols, collapse = ", ")
  )
}

dat2 <- dat %>%
  mutate(
    ResidueType = type,
    Event = suppressWarnings(as.numeric(.data[[event_col]])),
    Location = as.character(.data[[location_col]]),

    Result_chr = trimws(as.character(.data[[result_col_actual]])),
    ND_flag = is_nd_result(Result_chr),

    Result_raw = suppressWarnings(as.numeric(Result_chr)),
    LOD = suppressWarnings(as.numeric(.data[[lod_col]])),
    LOQ = suppressWarnings(as.numeric(.data[[loq_col]])),
    USL = suppressWarnings(as.numeric(.data[[usl_col]])),

    Result_Replacement_Source = case_when(
      ND_flag & !is.na(LOD) ~ "ND replaced by LOD",
      ND_flag & is.na(LOD) & !is.na(LOQ) ~ "ND replaced by LOQ",
      ND_flag & is.na(LOD) & is.na(LOQ) ~ "Removed: ND with missing LOD and LOQ",
      !ND_flag & !is.na(Result_raw) ~ "Observed Result",
      !ND_flag & is.na(Result_raw) ~ "Removed: missing or nonnumeric Result",
      TRUE ~ NA_character_
    ),

    Result_ND_Replaced = case_when(
      ND_flag & !is.na(LOD) ~ LOD,
      ND_flag & is.na(LOD) & !is.na(LOQ) ~ LOQ,

```

```

      ND_flag & is.na(LOD) & is.na(LOQ) ~ NA_real_,
      !ND_flag & !is.na(Result_raw) ~ Result_raw,
      !ND_flag & is.na(Result_raw) ~ NA_real_,
      TRUE ~ NA_real_
    )
  )

n_removed_nd_missing_lod_loq <- sum(
  dat2$ND_flag & is.na(dat2$LOD) & is.na(dat2$LOQ),
  na.rm = TRUE
)

n_removed_missing_result <- sum(
  !dat2$ND_flag & is.na(dat2$Result_raw),
  na.rm = TRUE
)

if (n_removed_nd_missing_lod_loq > 0) {
  warning(
    "Sheet ", type, ": ",
    n_removed_nd_missing_lod_loq,
    " ND row(s) were removed because both LOD and LOQ were missing."
  )
}

if (n_removed_missing_result > 0) {
  warning(
    "Sheet ", type, ": ",
    n_removed_missing_result,
    " row(s) were removed because Result was missing or nonnumeric."
  )
}

dat2 <- dat2 %>%
  filter(
    !(ND_flag & is.na(LOD) & is.na(LOQ)),
    !(ND_flag & is.na(Result_raw))
  )

if (any(is.na(dat2$USL))) {
  stop("Missing or nonnumeric USL detected in sheet ", type, ".")
}

if (any(dat2$USL <= 0, na.rm = TRUE)) {

```

```

    stop("USL must be positive in sheet ", type, ".")
  }

dat2 <- dat2 %>%
  mutate(
    Normalized = 100 * Result_ND_Replaced / USL
  )

if (any(is.na(dat2$Normalized))) {
  warning(
    "Missing normalized values detected in sheet ",
    type,
    ". Please check Result, LOD, LOQ, and USL."
  )
}

dat2 %>%
  select(
    ResidueType, Event, Location,
    Result_chr, Result_raw,
    LOD, LOQ, USL,
    ND_flag, Result_Replacement_Source,
    Result_ND_Replaced, Normalized
  )
}

normalized_data <- purrr::map_dfr(available_types, read_and_normalize)

# Step 3: Event-level medians

event_medians_long <- normalized_data %>%
  group_by(ResidueType, Event) %>%
  summarise(
    EventMedian = median(Normalized, na.rm = TRUE),
    .groups = "drop"
  )

event_medians_wide <- event_medians_long %>%
  tidyr::pivot_wider(
    id_cols = Event,
    names_from = ResidueType,
    values_from = EventMedian
  )

```

```

) %>%
  arrange(Event)

partially_missing_events <- event_medians_wide %>%
  filter(
    if_any(all_of(available_types), ~ is.na(.x)) &
    if_any(all_of(available_types), ~ !is.na(.x))
  )

if (nrow(partially_missing_events) > 0) {
  warning(
    "Some cleaning events have one or more missing residue-type medians. ",
    "These events will be retained, and the monitoring statistic will be ",
    "calculated using available residue-type values."
  )
}

events_all_missing <- event_medians_wide %>%
  filter(
    if_all(all_of(available_types), ~ is.na(.x))
  )

if (nrow(events_all_missing) > 0) {
  warning(
    "Some cleaning events have all available residue-type medians missing. ",
    "These events will be removed from monitoring statistic calculation."
  )
}

event_medians_wide_complete <- event_medians_wide %>%
  filter(
    if_any(all_of(available_types), ~ !is.na(.x))
  )

n_events <- nrow(event_medians_wide_complete)

if (n_events < 3) {
  stop(
    "Too few cleaning events with at least one available residue-type median ",
    "for Stage 3A SBM-L calibration."
  )
}

```

```

# Step 4: Global robust centers

global_centers <- event_medians_wide_complete %>%
  summarise(across(all_of(available_types), ~ median(.x, na.rm = TRUE)))

theta0 <- as.numeric(global_centers[1, available_types])
names(theta0) <- available_types

# Step 5: Observed monitoring statistics

observed_stats <- event_medians_wide_complete

for (type in available_types) {
  center_value <- theta0[type]
  observed_stats[[paste0("Dplus_", type)]] <- pmax(
    0,
    observed_stats[[type]] - center_value
  )
}

dplus_cols <- paste0("Dplus_", available_types)

observed_stats <- observed_stats %>%
  rowwise() %>%
  mutate(
    T_stat = max(c_across(all_of(dplus_cols)), na.rm = TRUE)
  ) %>%
  ungroup()

# Step 6: Location-stratified smoothed bootstrap

bootstrap_one_type <- function(type, dat_source = normalized_data) {

  dat_type <- dat_source %>%
    filter(ResidueType == type)

  locations <- sort(unique(dat_type$Location))
  boot_values <- numeric(length(locations))

```



```

for (m in seq_along(locations)) {

  loc <- locations[m]

  g_loc <- dat_type %>%
    filter(Location == loc) %>%
    pull(Normalized)

  g_loc <- g_loc[!is.na(g_loc)]

  if (length(g_loc) == 0) {
    boot_values[m] <- NA_real_
    next
  }

  med_g <- median(g_loc)
  mad_g <- median(abs(g_loc - med_g))
  h_loc <- c_smooth * mad_g

  sampled_value <- sample(g_loc, size = 1, replace = TRUE)
  eps <- rnorm(1, mean = 0, sd = h_loc)

  boot_values[m] <- max(0, sampled_value + eps)
}

median(boot_values, na.rm = TRUE)
}

generate_one_T <- function(dat_source = normalized_data) {

  boot_medians <- sapply(
    available_types,
    function(type) bootstrap_one_type(type, dat_source = dat_source)
  )

  boot_dplus <- pmax(0, boot_medians - theta0[available_types])

  max(boot_dplus, na.rm = TRUE)
}

T_boot <- numeric(B)

for (b in seq_len(B)) {
  T_boot[b] <- generate_one_T()
}

```

```

}

# Step 7: ARL0-based UCL

UCL_L <- as.numeric(
  quantile(T_boot, probs = 1 - alpha, na.rm = TRUE, type = 7)
)

empirical_p0 <- mean(T_boot > UCL_L, na.rm = TRUE)
empirical_ARL0 <- ifelse(empirical_p0 > 0, 1 / empirical_p0, Inf)

# Step 8: Monitoring rule and chart

chart_data <- observed_stats %>%
  mutate(
    UCL_L = UCL_L,
    Signal = T_stat > UCL_L
  )

p <- ggplot(chart_data, aes(x = Event, y = T_stat, group = 1)) +
  geom_line(color = "black") +
  geom_point(aes(color = Signal), size = 2.5) +
  geom_hline(
    yintercept = UCL_L,
    linetype = "dashed",
    color = "red",
    linewidth = 0.8
  ) +
  scale_color_manual(values = c("FALSE" = "black", "TRUE" = "red")) +
  labs(
    title = "Stage 3A SBM-L Control Chart",
    subtitle = paste0(
      "Residue types: ", paste(available_types, collapse = ", "),
      "; target ARL0 = ", target_ARL0,
      "; alpha = ", signif(alpha, 4),
      "; UCL = ", signif(UCL_L, 4)
    ),
    x = "Cleaning Event",
    y = expression(T[t]),
    color = "Signal"
  )

```

```

    ) +
    theme_bw()

    if (plot_chart) {
      print(p)
    }

    result <- list(
      file = file,
      available_types = available_types,
      target_ARL0 = target_ARL0,
      alpha = alpha,
      B = B,
      c_smooth = c_smooth,
      normalized_data = normalized_data,
      event_medians = event_medians_wide_complete,
      global_centers = theta0,
      observed_statistics = chart_data,
      bootstrap_statistics = T_boot,
      UCL_L = UCL_L,
      empirical_p0_from_calibration_bootstrap = empirical_p0,
      empirical_ARL0_from_calibration_bootstrap = empirical_ARL0,
      chart = p
    )

    return(result)
  }

```

```

res <- stage3A_SBM_L(
  file = file,
  target_ARL0 = 740,
  B = 10000,
  c_smooth = 0.5,
  seed = 123,
  plot_chart = FALSE
)

```

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = 740, B = 10000, c_smooth = 0.5, : Some cleaning events have one or more missing residue-type medians.

These events will be retained, and the monitoring statistic will be calculated using available residue-type values.

```
res$UCL_L
```

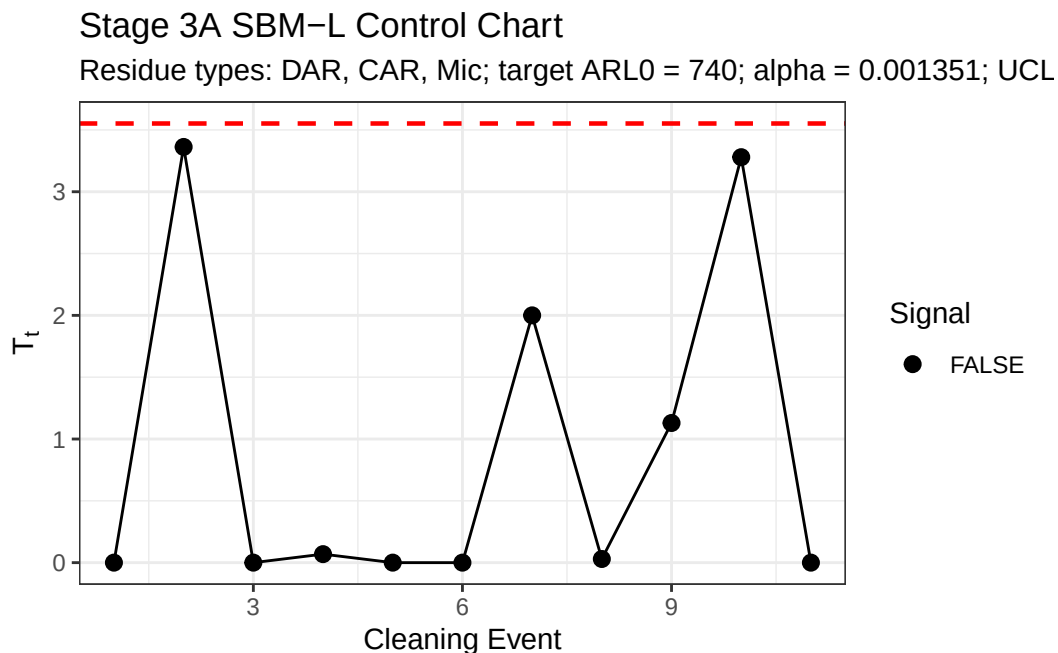
```
[1] 3.551968
```

```
res$observed_statistics
```

```
# A tibble: 11 x 10
```

	Event	CAR	DAR	Mic	Dplus_DAR	Dplus_CAR	Dplus_Mic	T_stat	UCL_L	Signal
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<lgl>
1	1	1.22	0.483	0	0	0	0	0	3.55	FALSE
2	2	4.08	3.88	0	3.36	2.86	0	3.36	3.55	FALSE
3	3	1.22	0.491	0	0	0	0	0	3.55	FALSE
4	4	1.22	0.589	0	0.0688	0	0	0.0688	3.55	FALSE
5	5	1.22	0.283	0	0	0	0	0	3.55	FALSE
6	6	1.22	0.0825	0	0	0	0	0	3.55	FALSE
7	7	1.22	0.183	2	0	0	2	2	3.55	FALSE
8	8	1.22	0.55	0	0.0295	0	0	0.0295	3.55	FALSE
9	9	1.22	1.65	0	1.13	0	0	1.13	3.55	FALSE
10	10	1.22	3.8	0	3.28	0	0	3.28	3.55	FALSE
11	11	1.22	NA	NA	NA	0	NA	0	3.55	FALSE

```
res$chart
```



1.1.1 C_smooth Function for Justification

```
test_c_for_SBM_L <- function(file,
                             c_grid = c(0, 0.25, 0.5, 1),
                             target_ARL0 = 740,
                             B = 10000,
                             seed = 123) {

  purrr::map_dfr(
    c_grid,
    function(c_val) {

      res_c <- stage3A_SBM_L(
        file = file,
        target_ARL0 = target_ARL0,
        B = B,
        c_smooth = c_val,
        seed = seed,
        plot_chart = FALSE
      )

      data.frame(
        c_smooth = c_val,
        target_ARL0 = res_c$target_ARL0,
        alpha = res_c$alpha,
        B = res_c$B,
        UCL_L = res_c$UCL_L,
        n_signal_observed = sum(res_c$observed_statistics$Signal, na.rm = TRUE),
        max_T_observed = max(res_c$observed_statistics$T_stat, na.rm = TRUE)
      )
    }
  )
}
```

1.1.2 c_smooth output

```
c_test <- test_c_for_SBM_L(
  file = file,
  c_grid = c(0, 0.25, 0.5, 1),
  target_ARL0 = 740,
  B = 10000,
```

```
seed = 123
)
```

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = target_ARL0, B = B, : Some cleaning events have one or more missing residue-type medians. These events will be retained, and the monitoring statistic will be calculated using available residue-type values.

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = target_ARL0, B = B, : Some cleaning events have one or more missing residue-type medians. These events will be retained, and the monitoring statistic will be calculated using available residue-type values.

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = target_ARL0, B = B, : Some cleaning events have one or more missing residue-type medians. These events will be retained, and the monitoring statistic will be calculated using available residue-type values.

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = target_ARL0, B = B, : Some cleaning events have one or more missing residue-type medians. These events will be retained, and the monitoring statistic will be calculated using available residue-type values.

```
c_test
```

	c_smooth	target_ARL0	alpha	B	UCL_L	n_signal_observed
1	0.00	740	0.001351351	10000	3.523527	0
2	0.25	740	0.001351351	10000	3.525121	0
3	0.50	740	0.001351351	10000	3.551968	0
4	1.00	740	0.001351351	10000	3.774077	0

	max_T_observed
1	3.36248
2	3.36248
3	3.36248
4	3.36248

The UCL was stable for $c_{\text{smooth}} = 0, 0.25$, and 0.5 , while $c_{\text{smooth}} = 1$ produced a larger UCL. Therefore, $c_{\text{smooth}} = 0.5$ was selected as the default value because it provides moderate smoothing without excessive UCL inflation.

1.1.3 Test B Function

```
library(dplyr)
library(purrr)

test_B_for_SBM_L <- function(file,
                              B_max = 30000,
                              B_grid = c(5000, 10000, 20000, 30000),
                              seeds = c(123, 456, 789),
                              target_ARL0 = 740,
                              c_smooth = 0.5) {

  alpha <- 1 / target_ARL0

  # Basic checks
  if (max(B_grid) > B_max) {
    stop("max(B_grid) must be <= B_max.")
  }

  all_results <- purrr::map_dfr(
    seeds,
    function(seed_val) {

      message("Running B_max = ", B_max, " with seed = ", seed_val)

      time_start <- Sys.time()

      res_max <- stage3A_SBM_L(
```

```

    file = file,
    target_ARL0 = target_ARL0,
    B = B_max,
    c_smooth = c_smooth,
    seed = seed_val,
    plot_chart = FALSE
  )

time_end <- Sys.time()
elapsed_sec <- as.numeric(difftime(time_end, time_start, units = "secs"))

T_boot <- res_max$bootstrap_statistics
T_obs <- res_max$observed_statistics$T_stat

purrr::map_dfr(
  B_grid,
  function(B_val) {

    T_sub <- T_boot[seq_len(B_val)]

    UCL_val <- as.numeric(
      quantile(
        T_sub,
        probs = 1 - alpha,
        na.rm = TRUE,
        type = 7
      )
    )

    p0_hat <- mean(T_sub > UCL_val, na.rm = TRUE)
    ARL0_hat <- ifelse(p0_hat > 0, 1 / p0_hat, Inf)

    data.frame(
      seed = seed_val,
      B = B_val,
      c_smooth = c_smooth,
      target_ARL0 = target_ARL0,
      alpha = alpha,
      expected_tail_count = B_val * alpha,
      UCL_L = UCL_val,
      empirical_p0 = p0_hat,
      empirical_ARL0 = ARL0_hat,
      n_signal_observed = sum(T_obs > UCL_val, na.rm = TRUE),
      max_T_observed = max(T_obs, na.rm = TRUE),

```



```

        elapsed_sec_for_Bmax_run = elapsed_sec
      )
    }
  )
}
)

summary_results <- all_results %>%
  group_by(B) %>%
  summarise(
    c_smooth = first(c_smooth),
    target_ARL0 = first(target_ARL0),
    alpha = first(alpha),
    expected_tail_count = first(expected_tail_count),
    mean_UCL = mean(UCL_L, na.rm = TRUE),
    sd_UCL = sd(UCL_L, na.rm = TRUE),
    min_UCL = min(UCL_L, na.rm = TRUE),
    max_UCL = max(UCL_L, na.rm = TRUE),
    range_UCL = max_UCL - min_UCL,
    mean_empirical_ARL0 = mean(empirical_ARL0, na.rm = TRUE),
    mean_n_signal_observed = mean(n_signal_observed, na.rm = TRUE),
    n_distinct_signal_counts = n_distinct(n_signal_observed),
    .groups = "drop"
  )

# Use largest B in grid as reference
B_ref <- max(B_grid)
UCL_ref <- summary_results %>%
  filter(B == B_ref) %>%
  pull(mean_UCL)

summary_results <- summary_results %>%
  mutate(
    abs_diff_from_ref_UCL = abs(mean_UCL - UCL_ref),
    rel_diff_from_ref_pct = 100 * abs(mean_UCL - UCL_ref) / UCL_ref
  )

list(
  raw_results = all_results,
  summary_results = summary_results
)
}

```

1.1.4 Test B Output and Conclusion

```
B_test <- test_B_for_SBM_L(  
  file = file,  
  B_max = 30000,  
  B_grid = c(5000, 10000, 20000, 30000),  
  seeds = c(123, 456, 789),  
  target_ARL0 = 740,  
  c_smooth = 0.5  
)
```

Running B_max = 30000 with seed = 123

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = target_ARL0, B = B_max, :
Some cleaning events have one or more missing residue-type medians. These
events will be retained, and the monitoring statistic will be calculated using
available residue-type values.

Running B_max = 30000 with seed = 456

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = target_ARL0, B = B_max, :
Some cleaning events have one or more missing residue-type medians. These
events will be retained, and the monitoring statistic will be calculated using
available residue-type values.

Running B_max = 30000 with seed = 789

Available residue types used in SBM-L chart: DAR, CAR, Mic

Target ARL0 = 740; alpha = 0.0013514

Warning in stage3A_SBM_L(file = file, target_ARL0 = target_ARL0, B = B_max, :
Some cleaning events have one or more missing residue-type medians. These
events will be retained, and the monitoring statistic will be calculated using
available residue-type values.

```
B_test$summary_results
```

```
# A tibble: 4 x 15
      B c_smooth target_ARL0 alpha expected_tail_count mean_UCL sd_UCL min_UCL
  <dbl>   <dbl>   <dbl>   <dbl>           <dbl>   <dbl> <dbl>   <dbl>
1  5000     0.5     740 0.00135           6.76     3.59 0.0617   3.54
2 10000     0.5     740 0.00135          13.5     3.57 0.0348   3.55
3 20000     0.5     740 0.00135          27.0     3.57 0.0230   3.55
4 30000     0.5     740 0.00135          40.5     3.59 0.0535   3.55
# i 7 more variables: max_UCL <dbl>, range_UCL <dbl>,
#   mean_empirical_ARL0 <dbl>, mean_n_signal_observed <dbl>,
#   n_distinct_signal_counts <int>, abs_diff_from_ref_UCL <dbl>,
#   rel_diff_from_ref_pct <dbl>
```

The UCL and observed monitoring decision were stable across the evaluated bootstrap sizes. Since $B = 10000$ produced results close to larger values of B while substantially reducing computation time, $B = 10000$ was selected for UCL calibration.

1.2 Estimate_ARL1_SBM_L Function

```
estimate_ARL1_SBM_L <- function(res,
                                shift = 1,
                                shifted_types = NULL,
                                shift_mode = c("additive", "multiplicative"),
                                B1 = 5000,
                                seed = 456) {

  shift_mode <- match.arg(shift_mode)

  library(dplyr)

  set.seed(seed)

  normalized_data <- res$normalized_data
  available_types <- res$available_types
  theta0 <- res$global_centers
  UCL_L <- res$UCL_L
  c_smooth <- res$c_smooth

  if (is.null(shifted_types)) {
    shifted_types <- available_types
```

```

}

shifted_types <- intersect(shifted_types, available_types)

if (length(shifted_types) == 0) {
  stop("No valid shifted_types were provided.")
}

# Create out-of-control shifted data

dat_shift <- normalized_data %>%
  mutate(
    Normalized_shifted = case_when(
      ResidueType %in% shifted_types & shift_mode == "additive" ~
        Normalized + shift,

      ResidueType %in% shifted_types & shift_mode == "multiplicative" ~
        Normalized * shift,

      TRUE ~ Normalized
    ),
    Normalized_shifted = pmax(0, Normalized_shifted)
  )

# Bootstrap one residue type under shifted condition

bootstrap_one_type_shift <- function(type) {

  dat_type <- dat_shift %>%
    filter(ResidueType == type)

  locations <- sort(unique(dat_type$Location))
  boot_values <- numeric(length(locations))

  for (m in seq_along(locations)) {

    loc <- locations[m]

    g_loc <- dat_type %>%
      filter(Location == loc) %>%

```

```

    pull(Normalized_shifted)

g_loc <- g_loc[!is.na(g_loc)]

if (length(g_loc) == 0) {
  boot_values[m] <- NA_real_
  next
}

med_g <- median(g_loc)
mad_g <- median(abs(g_loc - med_g))
h_loc <- c_smooth * mad_g

sampled_value <- sample(g_loc, size = 1, replace = TRUE)
eps <- rnorm(1, mean = 0, sd = h_loc)

boot_values[m] <- max(0, sampled_value + eps)
}

median(boot_values, na.rm = TRUE)
}

# Generate shifted SBM-L statistics

T_shift <- numeric(B1)

for (b in seq_len(B1)) {

  boot_medians <- sapply(
    available_types,
    bootstrap_one_type_shift
  )

  boot_dplus <- pmax(0, boot_medians - theta0[available_types])

  T_shift[b] <- max(boot_dplus, na.rm = TRUE)
}

# Estimate ARL1 and Monte Carlo precision

```

```

signal_indicator <- T_shift > UCL_L

n_signal_shift <- sum(signal_indicator, na.rm = TRUE)
p1_hat <- mean(signal_indicator, na.rm = TRUE)

ARL1_hat <- ifelse(p1_hat > 0, 1 / p1_hat, Inf)

# Monte Carlo standard error for p1_hat
se_p1 <- sqrt(p1_hat * (1 - p1_hat) / B1)

# Delta-method standard error for ARL1 = 1 / p1
se_ARL1 <- ifelse(
  p1_hat > 0,
  se_p1 / (p1_hat^2),
  Inf
)

relative_se_ARL1 <- ifelse(
  is.finite(ARL1_hat) & ARL1_hat > 0,
  se_ARL1 / ARL1_hat,
  Inf
)

ARL1_lower <- ifelse(
  is.finite(ARL1_hat),
  max(0, ARL1_hat - 1.96 * se_ARL1),
  Inf
)

# Sampling distribution is approximately normal
ARL1_upper <- ifelse(
  is.finite(ARL1_hat),
  ARL1_hat + 1.96 * se_ARL1,
  Inf
)

data.frame(
  c_smooth = c_smooth,
  target_ARL0 = res$target_ARL0,
  UCL_L = UCL_L,
  shift_mode = shift_mode,
  shift = shift,
  shifted_types = paste(shifted_types, collapse = ", "),
  B1 = B1,

```

```

    n_signal_shift = n_signal_shift,
    p1_hat = p1_hat,
    se_p1 = se_p1,
    ARL1_hat = ARL1_hat,
    se_ARL1 = se_ARL1,
    relative_se_ARL1 = relative_se_ARL1,
    ARL1_lower = ARL1_lower,
    ARL1_upper = ARL1_upper
  )
}

```

In this analysis, only additive shifts are used. The multiplicative option is retained in the function for flexibility but is not used in the reported ARL1 evaluation. On the USL-normalized scale, an additive shift of 1 corresponds to an increase of 1% of USL.

1.3 Test_B1_for_ARL1 Function

```

test_B1_for_ARL1 <- function(res,
                             B1_grid = c(1000, 3000, 5000, 10000, 20000),
                             seeds = c(123, 456, 789),
                             shift = 1,
                             shifted_types = c("DAR", "CAR", "Mic"),
                             shift_mode = "additive") {

  library(dplyr)
  library(purrr)

  all_results <- purrr::map_dfr(
    seeds,
    function(seed_val) {

      purrr::map_dfr(
        B1_grid,
        function(B1_val) {

          message(
            "Running ARL1 with B1 = ", B1_val,
            ", seed = ", seed_val
          )

          time_start <- Sys.time()

```

```

    out <- estimate_ARL1_SBM_L(
      res = res,
      shift = shift,
      shifted_types = shifted_types,
      shift_mode = shift_mode,
      B1 = B1_val,
      seed = seed_val
    )

    time_end <- Sys.time()

    out$seed <- seed_val
    out$elapsed_sec <- as.numeric(
      difftime(time_end, time_start, units = "secs")
    )

    out
  }
)
}
)

summary_results <- all_results %>%
  group_by(B1) %>%
  summarise(
    shift = first(shift),
    shifted_types = first(shifted_types),
    shift_mode = first(shift_mode),
    mean_n_signal_shift = mean(n_signal_shift, na.rm = TRUE),
    min_n_signal_shift = min(n_signal_shift, na.rm = TRUE),
    max_n_signal_shift = max(n_signal_shift, na.rm = TRUE),
    mean_p1_hat = mean(p1_hat, na.rm = TRUE),
    sd_p1_hat = sd(p1_hat, na.rm = TRUE),
    mean_ARL1 = mean(ARL1_hat, na.rm = TRUE),
    sd_ARL1 = sd(ARL1_hat, na.rm = TRUE),
    mean_se_ARL1 = mean(se_ARL1, na.rm = TRUE),
    mean_relative_se_ARL1 = mean(relative_se_ARL1, na.rm = TRUE),
    mean_ARL1_lower = mean(ARL1_lower, na.rm = TRUE),
    mean_ARL1_upper = mean(ARL1_upper, na.rm = TRUE),
    mean_elapsed_sec = mean(elapsed_sec, na.rm = TRUE),
    .groups = "drop"
  )

# Use largest B1 as reference

```



```

B1_ref <- max(B1_grid)

ARL1_ref <- summary_results %>%
  filter(B1 == B1_ref) %>%
  pull(mean_ARL1)

summary_results <- summary_results %>%
  mutate(
    abs_diff_from_ref_ARL1 = abs(mean_ARL1 - ARL1_ref),
    rel_diff_from_ref_ARL1_pct =
      100 * abs(mean_ARL1 - ARL1_ref) / ARL1_ref
  )

list(
  raw_results = all_results,
  summary_results = summary_results
)
}

```

1.3.1 B1 Test Output and Conclusion

```

B1_test <- test_B1_for_ARL1(
  res = res,
  B1_grid = c(1000, 3000, 5000, 10000, 20000),
  seeds = c(123, 456, 789),
  shift = 1,
  shifted_types = c("DAR", "CAR", "Mic"),
  shift_mode = "additive"
)

```

Running ARL1 with B1 = 1000, seed = 123

Running ARL1 with B1 = 3000, seed = 123

Running ARL1 with B1 = 5000, seed = 123

Running ARL1 with B1 = 10000, seed = 123

Running ARL1 with B1 = 20000, seed = 123

Running ARL1 with B1 = 1000, seed = 456

Running ARL1 with B1 = 3000, seed = 456

Running ARL1 with B1 = 5000, seed = 456

Running ARL1 with B1 = 10000, seed = 456

Running ARL1 with B1 = 20000, seed = 456

Running ARL1 with B1 = 1000, seed = 789

Running ARL1 with B1 = 3000, seed = 789

Running ARL1 with B1 = 5000, seed = 789

Running ARL1 with B1 = 10000, seed = 789

Running ARL1 with B1 = 20000, seed = 789

```
print(B1_test$summary_results, width = Inf)
```

A tibble: 5 x 18

	B1	shift	shifted_types	shift_mode	mean_n_signal_shift	min_n_signal_shift
	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<int>
1	1000	1	DAR, CAR, Mic	additive	22	19
2	3000	1	DAR, CAR, Mic	additive	74.7	73
3	5000	1	DAR, CAR, Mic	additive	120.	104
4	10000	1	DAR, CAR, Mic	additive	237	225
5	20000	1	DAR, CAR, Mic	additive	489	481
	max_n_signal_shift	mean_p1_hat	sd_p1_hat	mean_ARL1	sd_ARL1	mean_se_ARL1
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	27	0.022	0.00436	46.6	8.35	10.0
2	78	0.0249	0.000962	40.2	1.52	4.60
3	133	0.0241	0.00297	42.0	5.44	3.82
4	245	0.0237	0.00106	42.3	1.93	2.72
5	497	0.0244	0.000400	40.9	0.669	1.83
	mean_relative_se_ARL1	mean_ARL1_lower	mean_ARL1_upper	mean_elapsed_sec		
	<dbl>	<dbl>	<dbl>	<dbl>		
1	0.213	26.9	66.2	64.9		
2	0.114	31.2	49.2	192.		
3	0.0904	34.5	49.5	329.		
4	0.0642	36.9	47.6	648.		

5	0.0447	37.3	44.5	1301.
	abs_diff_from_ref_ARL1	rel_diff_from_ref_ARL1_pct		
	<dbl>	<dbl>		
1	5.65	13.8		
2	0.689	1.69		
3	1.09	2.67		
4	1.34	3.29		
5	0	0		

```
#B1_test$summary_results
```

```
arl1_shift_1 <- estimate_ARL1_SBM_L(
  res = res,
  shift = 1,
  shifted_types = c("DAR", "CAR", "Mic"),
  shift_mode = "additive",
  B1 = 5000,
  seed = 456
)

arl1_shift_1
```

	c_smooth	target_ARL0	UCL_L	shift_mode	shift	shifted_types	B1
1	0.5	740	3.551968	additive	1	DAR, CAR, Mic	5000
	n_signal_shift	p1_hat	se_p1	ARL1_hat	se_ARL1	relative_se_ARL1	
1	124	0.0248	0.002199316	40.32258	3.575891	0.08868211	
	ARL1_lower	ARL1_upper					
1	33.31383	47.33133					

For a global additive shift of +1% USL across DAR, CAR, and Mic, the estimated ARL1 was obtained using $B1 = 5000$. Although this shift is small relative to the USL, it may represent a meaningful increase relative to the observed baseline residue levels.

```
shift_grid <- c(0.25, 0.5, 1, 2, 3, 5)

arl1_grid <- purrr::map_dfr(
  shift_grid,
  function(delta) {
    estimate_ARL1_SBM_L(
      res = res,
      shift = delta,
      shifted_types = c("DAR", "CAR", "Mic"),
      shift_mode = "additive",
```

```

    B1 = 5000,
    seed = 456
  )
}
)

arl1_grid

```

	c_smooth	target_ARL0	UCL_L	shift_mode	shift	shifted_types	B1
1	0.5	740	3.551968	additive	0.25	DAR, CAR, Mic	5000
2	0.5	740	3.551968	additive	0.50	DAR, CAR, Mic	5000
3	0.5	740	3.551968	additive	1.00	DAR, CAR, Mic	5000
4	0.5	740	3.551968	additive	2.00	DAR, CAR, Mic	5000
5	0.5	740	3.551968	additive	3.00	DAR, CAR, Mic	5000
6	0.5	740	3.551968	additive	5.00	DAR, CAR, Mic	5000

	n_signal_shift	p1_hat	se_p1	ARL1_hat	se_ARL1	relative_se_ARL1
1	26	0.0052	0.001017149	192.307692	37.6164555	0.19560557
2	67	0.0134	0.001626065	74.626866	9.0558320	0.12134815
3	124	0.0248	0.002199316	40.322581	3.5758914	0.08868211
4	556	0.1112	0.004446000	8.992806	0.3595505	0.03998201
5	1205	0.2410	0.006048454	4.149378	0.1041383	0.02509732
6	5000	1.0000	0.000000000	1.000000	0.0000000	0.00000000

	ARL1_lower	ARL1_upper
1	118.579440	266.035945
2	56.877435	92.376296
3	33.313833	47.331328
4	8.288087	9.697525
5	3.945267	4.353489
6	1.000000	1.000000

The ARL1 estimates stabilized for $B1 \geq 5000$. At $B1 = 5000$, the average number of shifted signals was adequate and the relative Monte Carlo standard error was below 10%. Therefore, $B1 = 5000$ was selected for ARL1 estimation to balance precision and computational efficiency.